



Chapter 7. *Cancellation and Shutdown*

It is easy to start tasks and threads. Most of the time we allow them to decide when to stop by letting them run to completion. Sometimes, however, we want to stop tasks or threads earlier than they would on their own, perhaps because the user cancelled an operation or the application needs to shut down quickly.

Getting tasks and threads to stop safely, quickly, and reliably is not always easy. Java does not provide any mechanism for safely forcing a thread to stop what it is doing.^[1] Instead, it provides *interruption*, a cooperative mechanism that lets one thread ask another to stop what it is doing.

[1] The deprecated `Thread.stop` and `suspend` methods were an attempt to provide such a mechanism, but were quickly realized to be seriously flawed and should be avoided. See

<http://java.sun.com/j2se/1.5.0/docs/guide/misc/threadPrimitiveDeprecation.html> for an explanation of the problems with these methods.

The cooperative approach is required because we rarely want a task, thread, or service to stop *immediately*, since that could leave shared data structures in an inconsistent state. Instead, tasks and services can be coded so that, when requested, they clean up any work currently in progress and *then* terminate. This provides greater flexibility, since the task code itself is usually better able to assess the cleanup required than is the code requesting cancellation.

End-of-lifecycle issues can complicate the design and implementation of tasks, services, and applications, and this important element of program design is too often ignored. Dealing well with failure, shutdown, and cancellation is one of the characteristics that distinguishes a well-behaved application from one that merely works. This chapter addresses mecha-

nisms for cancellation and interruption, and how to code tasks and services to be responsive to cancellation requests.

7.1. Task Cancellation

An activity is *cancellable* if external code can move it to completion before its normal completion. There are a number of reasons why you might want to cancel an activity:

User-requested cancellation. The user clicked on the “cancel” button in a GUI application, or requested cancellation through a management interface such as JMX (Java Management Extensions).

Time-limited activities. An application searches a problem space for a finite amount of time and chooses the best solution found within that time. When the timer expires, any tasks still searching are cancelled.

Application events. An application searches a problem space by decomposing it so that different tasks search different regions of the problem space. When one task finds a solution, all other tasks still searching are cancelled.

Errors. A web crawler searches for relevant pages, storing pages or summary data to disk. When a crawler task encounters an error (for example, the disk is full), other crawling tasks are cancelled, possibly recording their current state so that they can be restarted later.

Shutdown. When an application or service is shut down, something must be done about work that is currently being processed or queued for processing. In a graceful shutdown, tasks currently in progress might be allowed to complete; in a more immediate shutdown, currently executing tasks might be cancelled.

There is no safe way to preemptively stop a thread in Java, and therefore no safe way to preemptively stop a task. There are only cooperative mechanisms, by which the task and the code requesting cancellation follow an agreed-upon protocol.

One such cooperative mechanism is setting a “cancellation requested” flag that the task checks periodically; if it finds the flag set, the task terminates early. `PrimeGenerator` in [Listing 7.1](#), which enumerates prime numbers until it is cancelled, illustrates this technique. The `cancel` method sets the `cancelled` flag, and the main loop polls this flag before searching for the next prime number. (For this to work reliably, `cancelled` must be `volatile`.)

[Listing 7.2](#) shows a sample use of this class that lets the prime generator run for one second before cancelling it. The generator won't necessarily stop after exactly one second, since there may be some delay between the time that cancellation is requested and the time that the `run` loop next checks for cancellation. The `cancel` method is called from a `finally` block to ensure that the prime generator is cancelled even if the the call to `sleep` is interrupted. If `cancel` were not called, the prime-seeking thread would run forever, consuming CPU cycles and preventing the JVM from exiting.

A task that wants to be cancellable must have a *cancellation policy* that specifies the “how”, “when”, and “what” of cancellation—how other code can request cancellation, when the task checks whether cancellation has been requested, and what actions the task takes in response to a cancellation request.

Consider the real-world example of stopping payment on a check. Banks have rules about how to submit a stop-payment request, what responsiveness guarantees it makes in processing such requests, and what procedures it follows when payment is actually stopped (such as notifying the other bank involved in the transaction and assessing a fee against the payor's account). Taken together, these procedures and guarantees comprise the cancellation policy for check payment.

Listing 7.1. Using a `volatile` Field to Hold Cancellation State.

```

@ThreadSafe
public class PrimeGenerator implements Runnable {
    @GuardedBy("this")
    private final List<BigInteger> primes
        = new ArrayList<BigInteger>();
    private volatile boolean cancelled;

    public void run() {
        BigInteger p = BigInteger.ONE;
        while (!cancelled) {
            p = p.nextProbablePrime();
            synchronized (this) {
                primes.add(p);
            }
        }
    }

    public void cancel() { cancelled = true; }

    public synchronized List<BigInteger> get() {
        return new ArrayList<BigInteger>(primes);
    }
}

```

Listing 7.2. Generating a Second's Worth of Prime Numbers.

```

List<BigInteger> aSecondOfPrimes() throws InterruptedException {
    PrimeGenerator generator = new PrimeGenerator();
    new Thread(generator).start();
    try {
        SECONDS.sleep(1);
    } finally {
        generator.cancel();
    }
    return generator.get();
}

```

`PrimeGenerator` uses a simple cancellation policy: client code requests cancellation by calling `cancel`, `PrimeGenerator` checks for cancellation once per prime found and exits when it detects cancellation has been requested.

7.1.1. Interruption

The cancellation mechanism in `PrimeGenerator` will eventually cause the primeseeking task to exit, but it might take a while. If, however, a task that uses this approach calls a blocking method such as

`BlockingQueue.put`, we could have a more serious problem—the task might never check the cancellation flag and therefore might never terminate.

`BrokenPrimeProducer` in [Listing 7.3](#) illustrates this problem. The producer thread generates primes and places them on a blocking queue. If the producer gets ahead of the consumer, the queue will fill up and `put` will block. What happens if the consumer tries to cancel the producer task while it is blocked in `put`? It can call `cancel` which will set the `cancelled` flag—but the producer will never check the flag because it will never emerge from the blocking `put` (because the consumer has stopped retrieving primes from the queue).

As we hinted in [Chapter 5](#), certain blocking library methods support *interruption*. Thread interruption is a cooperative mechanism for a thread to signal another thread that it should, at its convenience and if it feels like it, stop what it is doing and do something else.

There is nothing in the API or language specification that ties interruption to any specific cancellation semantics, but in practice, using interruption for anything but cancellation is fragile and difficult to sustain in larger applications.

Each thread has a boolean *interrupted status*; interrupting a thread sets its interrupted status to true. `Thread` contains methods for interrupting a thread and querying the interrupted status of a thread, as shown in [Listing 7.4](#). The `interrupt` method interrupts the target thread, and `isInterrupted` returns the interrupted status of the target thread. The poorly named static `interrupted` method *clears* the interrupted status of the current thread and returns its previous value; this is the only way to clear the interrupted status.

Blocking library methods like `Thread.sleep` and `Object.wait` try to detect when a thread has been interrupted and return early. They respond to interruption by clearing the interrupted status and throwing `InterruptedException`, indicating that the blocking operation com-

pleted early due to interruption. The JVM makes no guarantees on how quickly a blocking method will detect interruption, but in practice this happens reasonably quickly.

Listing 7.3. Unreliable Cancellation that can Leave Producers Stuck in a Blocking Operation. *Don't do this.*

```
class BrokenPrimeProducer extends Thread {
    private final BlockingQueue<BigInteger> queue;
    private volatile boolean cancelled = false;

    BrokenPrimeProducer(BlockingQueue<BigInteger> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
            BigInteger p = BigInteger.ONE;
            while (!cancelled)
                queue.put(p = p.nextProbablePrime());
        } catch (InterruptedException consumed) { }
    }

    public void cancel() { cancelled = true; }
}

void consumePrimes() throws InterruptedException {
    BlockingQueue<BigInteger> primes = ...;
    BrokenPrimeProducer producer = new BrokenPrimeProducer(primes);
    producer.start();
    try {
        while (needMorePrimes())
            consume(primes.take());
    } finally {
        producer.cancel();
    }
}
```



Listing 7.4. Interruption Methods in Thread.

```
public class Thread {
    public void interrupt() { ... }
    public boolean isInterrupted() { ... }
    public static boolean interrupted() { ... }
    ...
}
```

If a thread is interrupted when it is *not* blocked, its interrupted status is set, and it is up to the activity being cancelled to poll the interrupted status to detect interruption. In this way interruption is “sticky”—if it doesn't

trigger an `InterruptedException`, evidence of interruption persists until someone deliberately clears the interrupted status.

Calling `interrupt` does not necessarily stop the target thread from doing what it is doing; it merely delivers the message that interruption has been requested.

A good way to think about interruption is that it does not actually interrupt a running thread; it just *requests* that the thread interrupt itself at the next convenient opportunity. (These opportunities are called *cancellation points*.) Some methods, such as `wait`, `sleep`, and `join`, take such requests seriously, throwing an exception when they receive an interrupt request or encounter an already set interrupt status upon entry. Well behaved methods may totally ignore such requests so long as they leave the interruption request in place so that calling code can do something with it. Poorly behaved methods swallow the interrupt request, thus denying code further up the call stack the opportunity to act on it.

The static `interrupted` method should be used with caution, because it clears the current thread's interrupted status. If you call `interrupted` and it returns `true`, unless you are planning to swallow the interruption, you should do something with it—either throw `InterruptedException` or restore the interrupted status by calling `interrupt` again, as in [Listing 5.10](#) on page [94](#).

`BrokenPrimeProducer` illustrates how custom cancellation mechanisms do not always interact well with blocking library methods. If you code your tasks to be responsive to interruption, you can use interruption as your cancellation mechanism and take advantage of the interruption support provided by many library classes.

Interruption is usually the most sensible way to implement cancellation.

`BrokenPrimeProducer` can be easily fixed (and simplified) by using interruption instead of a boolean flag to request cancellation, as shown in

Listing 7.5. There are two points in each loop iteration where interruption may be detected: in the blocking `put` call, and by explicitly polling the interrupted status in the loop header. The explicit test is not strictly necessary here because of the blocking `put` call, but it makes `PrimeProducer` more responsive to interruption because it checks for interruption *before* starting the lengthy task of searching for a prime, rather than after. When calls to interruptible blocking methods are not frequent enough to deliver the desired responsiveness, explicitly testing the interrupted status can help.

Listing 7.5. Using Interruption for Cancellation.

```
class PrimeProducer extends Thread {
    private final BlockingQueue<BigInteger> queue;

    PrimeProducer(BlockingQueue<BigInteger> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
            BigInteger p = BigInteger.ONE;
            while (!Thread.currentThread().isInterrupted())
                queue.put(p = p.nextProbablePrime());
        } catch (InterruptedException consumed) {
            /* Allow thread to exit */
        }
    }

    public void cancel() { interrupt(); }
}
```

7.1.2. Interruption Policies

Just as tasks should have a cancellation policy, threads should have an *interruption policy*. An interruption policy determines how a thread interprets an interruption request—what it does (if anything) when one is detected, what units of work are considered atomic with respect to interruption, and how quickly it reacts to interruption.

The most sensible interruption policy is some form of thread-level or servicelevel cancellation: exit as quickly as practical, cleaning up if necessary, and possibly notifying some owning entity that the thread is exiting. It is possible to establish other interruption policies, such as pausing or

resuming a service, but threads or thread pools with nonstandard interruption policies may need to be restricted to tasks that have been written with an awareness of the policy.

It is important to distinguish between how *tasks* and *threads* should react to interruption. A single interrupt request may have more than one desired recipient—interrupting a worker thread in a thread pool can mean both “cancel the current task” and “shut down the worker thread”.

Tasks do not execute in threads they own; they borrow threads owned by a service such as a thread pool. Code that doesn't own the thread (for a thread pool, any code outside of the thread pool implementation) should be careful to preserve the interrupted status so that the owning code can eventually act on it, even if the “guest” code acts on the interruption as well. (If you are house-sitting for someone, you don't throw out the mail that comes while they're away—you save it and let them deal with it when they get back, even if you do read their magazines.)

This is why most blocking library methods simply throw `InterruptedException` in response to an interrupt. They will never execute in a thread they own, so they implement the most reasonable cancellation policy for task or library code: get out of the way as quickly as possible and communicate the interruption back to the caller so that code higher up on the call stack can take further action.

A task needn't necessarily drop everything when it detects an interruption request—it can choose to postpone it until a more opportune time by remembering that it was interrupted, finishing the task it was performing, and *then* throwing `InterruptedException` or otherwise indicating interruption. This technique can protect data structures from corruption when an activity is interrupted in the middle of an update.

A task should not assume anything about the interruption policy of its executing thread unless it is explicitly designed to run within a service that has a specific interruption policy. Whether a task interprets interruption as cancellation or takes some other action on interruption, it should take care to preserve the executing thread's interruption status. If it is not simply going to propagate `InterruptedException` to its caller, it should

restore the interruption status after catching `InterruptedException` :

```
Thread.currentThread().interrupt();
```

Just as task code should not make assumptions about what interruption means to its executing thread, cancellation code should not make assumptions about the interruption policy of arbitrary threads. A thread should be interrupted only by its owner; the owner can encapsulate knowledge of the thread's interruption policy in an appropriate cancellation mechanism such as a shutdown method.

Because each thread has its own interruption policy, you should not interrupt a thread unless you know what interruption means to that thread.

Critics have derided the Java interruption facility because it does not provide a preemptive interruption capability and yet forces developers to handle `InterruptedException`. However, the ability to postpone an interruption request enables developers to craft flexible interruption policies that balance responsiveness and robustness as appropriate for the application.

7.1.3. Responding to Interruption

As mentioned in [Section 5.4](#), when you call an interruptible blocking method such as `Thread.sleep` or `BlockingQueue.put`, there are two practical strategies for handling `InterruptedException` :

- Propagate the exception (possibly after some task-specific cleanup), making your method an interruptible blocking method, too; or
- Restore the interruption status so that code higher up on the call stack can deal with it.

Propagating `InterruptedException` can be as easy as adding `InterruptedException` to the `throws` clause, as shown by `getNextTask` in [Listing 7.6](#).

Listing 7.6. Propagating `InterruptedException` to Callers.

```
BlockingQueue<Task> queue;  
...  
public Task getNextTask() throws InterruptedException {  
    return queue.take();  
}
```

If you don't want to or cannot propagate `InterruptedException` (perhaps because your task is defined by a `Runnable`), you need to find another way to preserve the interruption request. The standard way to do this is to restore the interrupted status by calling `interrupt` again.

What you should *not* do is swallow the `InterruptedException` by catching it and doing nothing in the `catch` block, unless your code is actually implementing the interruption policy for a thread.

`PrimeProducer` swallows the interrupt, but does so with the knowledge that the thread is about to terminate and that therefore there is no code higher up on the call stack that needs to know about the interruption. Most code does not know what thread it will run in and so should preserve the interrupted status.

Only code that implements a thread's interruption policy may swallow an interruption request. General-purpose task and library code should never swallow interruption requests.

Activities that do not support cancellation but still call interruptible blocking methods will have to call them in a loop, retrying when interruption is detected. In this case, they should save the interruption status locally and restore it just before returning, as shown in [Listing 7.7](#), rather than immediately upon catching `InterruptedException`. Setting the interrupted status too early could result in an infinite loop, because most interruptible blocking methods check the interrupted status on entry and throw `InterruptedException` immediately if it is set. (Interruptible methods usually poll for interruption before blocking or doing any significant work, so as to be as responsive to interruption as possible.)

If your code does not call interruptible blocking methods, it can still be made responsive to interruption by polling the current thread's interrupted status throughout the task code. Choosing a polling frequency is a

tradeoff between efficiency and responsiveness. If you have high responsiveness requirements, you cannot call potentially long-running methods that are not themselves responsive to interruption, potentially restricting your options for calling library code.

Cancellation can involve state other than the interruption status; interruption can be used to get the thread's attention, and information stored elsewhere by the interrupting thread can be used to provide further instructions for the interrupted thread. (Be sure to use synchronization when accessing that information.)

Listing 7.7. Noncancelable Task that Restores Interruption Before Exit.

```
public Task getNextTask(BlockingQueue<Task> queue) {
    boolean interrupted = false;
    try {
        while (true) {
            try {
                return queue.take();
            } catch (InterruptedException e) {
                interrupted = true;
                // fall through and retry
            }
        }
    } finally {
        if (interrupted)
            Thread.currentThread().interrupt();
    }
}
```

For example, when a worker thread owned by a `ThreadPoolExecutor` detects interruption, it checks whether the pool is being shut down. If so, it performs some pool cleanup before terminating; otherwise it may create a new thread to restore the thread pool to the desired size.

7.1.4. Example: Timed Run

Many problems can take forever to solve (e.g., enumerate all the prime numbers); for others, the answer might be found reasonably quickly but also might take forever. Being able to say “spend up to ten minutes looking for the answer” or “enumerate all the answers you can in ten minutes” can be useful in these situations.

The `aSecondOfPrimes` method in [Listing 7.2](#) starts a `PrimeGenerator` and interrupts it after a second. While the `PrimeGenerator` might take somewhat longer than a second to stop, it will eventually notice the interrupt and stop, allowing the thread to terminate. But another aspect of executing a task is that you want to find out if the task throws an exception. If `PrimeGenerator` throws an unchecked exception before the timeout expires, it will probably go unnoticed, since the prime generator runs in a separate thread that does not explicitly handle exceptions.

[Listing 7.8](#) shows an attempt at running an arbitrary `Runnable` for a given amount of time. It runs the task in the calling thread and schedules a cancellation task to interrupt it after a given time interval. This addresses the problem of unchecked exceptions thrown from the task, since they can then be caught by the caller of `timedRun`.

This is an appealingly simple approach, but it violates the rules: you should know a thread's interruption policy before interrupting it. Since `timedRun` can be called from an arbitrary thread, it cannot know the calling thread's interruption policy. If the task completes before the timeout, the cancellation task that interrupts the thread in which `timedRun` was called could go off *after* `timedRun` has returned to its caller. We don't know what code will be running when that happens, but the result won't be good. (It is possible but surprisingly tricky to eliminate this risk by using the `ScheduledFuture` returned by `schedule` to cancel the cancellation task.)

Listing 7.8. Scheduling an Interrupt on a Borrowed Thread. *Don't do this.*

```
private static final ScheduledExecutorService cancelExec = ...;

public static void timedRun(Runnable r,
                           long timeout, TimeUnit unit) {
    final Thread taskThread = Thread.currentThread();
    cancelExec.schedule(new Runnable() {
        public void run() { taskThread.interrupt(); }
    }, timeout, unit);
    r.run();
}
```



Further, if the task is not responsive to interruption, `timedRun` will not return until the task finishes, which may be long after the desired timeout (or even not at all). A timed run service that doesn't return after the specified time is likely to be irritating to its callers.

Listing 7.9 addresses the exception-handling problem of `aSecondOfPrimes` and the problems with the previous attempt. The thread created to run the task can have its own execution policy, and even if the task doesn't respond to the interrupt, the timed run method can still return to its caller. After starting the task thread, `timedRun` executes a timed `join` with the newly created thread. After `join` returns, it checks if an exception was thrown from the task and if so, rethrows it in the thread calling `timedRun`. The saved `Throwable` is shared between the two threads, and so is declared `volatile` to safely publish it from the task thread to the `timedRun` thread.

This version addresses the problems in the previous examples, but because it relies on a timed `join`, it shares a deficiency with `join`: we don't know if control was returned because the thread exited normally or because the `join` timed out.^[2]

[2] This is a flaw in the `Thread` API, because whether or not the `join` completes successfully has memory visibility consequences in the Java Memory Model, but `join` does not return a status indicating whether it was successful.

7.1.5. Cancellation Via `Future`

We've already used an abstraction for managing the lifecycle of a task, dealing with exceptions, and facilitating cancellation—`Future`. Following the general principle that it is better to use existing library classes than to roll your own, let's build `timedRun` using `Future` and the task execution framework.

Listing 7.9. Interrupting a Task in a Dedicated Thread.


```

public static void timedRun(final Runnable r,
                           long timeout, TimeUnit unit)
    throws InterruptedException {
    class RethrowableTask implements Runnable {
        private volatile Throwable t;
        public void run() {
            try { r.run(); }
            catch (Throwable t) { this.t = t; }
        }
        void rethrow() {
            if (t != null)
                throw launderThrowable(t);
        }
    }

    RethrowableTask task = new RethrowableTask();
    final Thread taskThread = new Thread(task);
    taskThread.start();
    cancelExec.schedule(new Runnable() {
        public void run() { taskThread.interrupt(); }
    }, timeout, unit);
    taskThread.join(unit.toMillis(timeout));
    task.rethrow();
}

```



`ExecutorService.submit` returns a `Future` describing the task. `Future` has a `cancel` method that takes a boolean argument, `mayInterruptIfRunning`, and returns a value indicating whether the cancellation attempt was successful. (This tells you only whether it was able to deliver the interruption, not whether the task detected and acted on it.) When `mayInterruptIfRunning` is `true` and the task is currently running in some thread, then that thread is interrupted. Setting this argument to `false` means “don't run this task if it hasn't started yet”, and should be used for tasks that are not designed to handle interruption.

Since you shouldn't interrupt a thread unless you know its interruption policy, when is it OK to call `cancel` with an argument of `true`? The task execution threads created by the standard `Executor` implementations implement an interruption policy that lets tasks be cancelled using interruption, so it is safe to set `mayInterruptIfRunning` when cancelling tasks through their `Futures` when they are running in a standard `Executor`. You should not interrupt a pool thread directly when attempting to cancel a task, because you won't know what task is running when the interrupt request is delivered—do this only through the task's `Future`. This is yet another reason to code tasks to treat interruption as

a cancellation request: then they can be cancelled through their `Future`s.

Listing 7.10 shows a version of `timedRun` that submits the task to an `ExecutorService` and retrieves the result with a timed `Future.get`. If `get` terminates with a `TimeoutException`, the task is cancelled via its `Future`. (To simplify coding, this version calls `Future.cancel` unconditionally in a `finally` block, taking advantage of the fact that cancelling a completed task has no effect.) If the underlying computation throws an exception prior to cancellation, it is rethrown from `timedRun`, which is the most convenient way for the caller to deal with the exception. **Listing 7.10** also illustrates another good practice: cancelling tasks whose result is no longer needed. (This technique was also used in **Listing 6.13** on page 128 and **Listing 6.16** on page 132.)

Listing 7.10. Cancelling a Task Using `Future`.

```
public static void timedRun(Runnable r,
                           long timeout, TimeUnit unit)
    throws InterruptedException {
    Future<?> task = taskExec.submit(r);
    try {
        task.get(timeout, unit);
    } catch (TimeoutException e) {
        // task will be cancelled below
    } catch (ExecutionException e) {
        // exception thrown in task; rethrow
        throw launderThrowable(e.getCause());
    } finally {
        // Harmless if task already completed
        task.cancel(true); // interrupt if running
    }
}
```

When `Future.get` throws `InterruptedException` or `TimeoutException` and you know that the result is no longer needed by the program, cancel the task with `Future.cancel`.

7.1.6. Dealing with Non-interruptible Blocking

Many blocking library methods respond to interruption by returning early and throwing `InterruptedException`, which makes it easier to build tasks that are responsive to cancellation. However, not all blocking methods or blocking mechanisms are responsive to interruption; if a thread is blocked performing synchronous socket I/O or waiting to acquire an intrinsic lock, interruption has no effect other than setting the thread's interrupted status. We can sometimes convince threads blocked in noninterruptible activities to stop by means similar to interruption, but this requires greater awareness of why the thread is blocked.

Synchronous socket I/O in `java.io`. The common form of blocking I/O in server applications is reading or writing to a socket. Unfortunately, the `read` and `write` methods in `InputStream` and `OutputStream` are not responsive to interruption, but closing the underlying socket makes any threads blocked in `read` or `write` throw a `SocketException`.

Synchronous I/O in `java.nio`. Interrupting a thread waiting on an `InterruptibleChannel` causes it to throw `ClosedByInterruptException` and close the channel (and also causes all other threads blocked on the channel to throw `ClosedByInterruptException`). Closing an `InterruptibleChannel` causes threads blocked on channel operations to throw `AsynchronousCloseException`. Most standard `Channels` implement `InterruptibleChannel`.

Asynchronous I/O with `Selector`. If a thread is blocked in `Selector.select` (in `java.nio.channels`), calling `close` or `wakeup` causes it to return prematurely.

Lock acquisition. If a thread is blocked waiting for an intrinsic lock, there is nothing you can do to stop it short of ensuring that it eventually acquires the lock and makes enough progress that you can get its attention some other way. However, the explicit `Lock` classes offer the `lockInterruptibly` method, which allows you to wait for a lock and still be responsive to interrupts—see [Chapter 13](#).

`ReaderThread` in [Listing 7.11](#) shows a technique for encapsulating non-standard cancellation. `ReaderThread` manages a single socket connec-

tion, reading synchronously from the socket and passing any data received to `processBuffer`. To facilitate terminating a user connection or shutting down the server, `ReaderThread` overrides `interrupt` to both deliver a standard interrupt and close the underlying socket; thus interrupting a `ReaderThread` makes it stop what it is doing whether it is blocked in `read` or in an interruptible blocking method.

7.1.7. Encapsulating nonstandard cancellation with `newTaskFor`

The technique used in `ReaderThread` to encapsulate nonstandard cancellation can be refined using the `newTaskFor` hook added to `ThreadPoolExecutor` in Java 6. When a `Callable` is submitted to an `ExecutorService`, `submit` returns a `Future` that can be used to cancel the task. The `newTaskFor` hook is a factory method that creates the `Future` representing the task. It returns a `RunnableFuture`, an interface that extends both `Future` and `Runnable` (and is implemented by `FutureTask`).

Customizing the task `Future` allows you to override `Future.cancel`. Custom cancellation code can perform logging or gather statistics on cancellation, and can also be used to cancel activities that are not responsive to interruption. `ReaderThread` encapsulates cancellation of socket-using threads by overriding `interrupt`; the same can be done for tasks by overriding `Future.cancel`.

`CancellableTask` in [Listing 7.12](#) defines a `CancellableTask` interface that extends `Callable` and adds a `cancel` method and a `newTask` factory method for constructing a `RunnableFuture`.

`CancellingExecutor` extends `ThreadPoolExecutor`, and overrides `newTaskFor` to let a `CancellableTask` create its own `Future`.

Listing 7.11. Encapsulating Nonstandard Cancellation in a Thread by Overriding `Interrupt`.

```

public class ReaderThread extends Thread {
    private final Socket socket;
    private final InputStream in;

    public ReaderThread(Socket socket) throws IOException {
        this.socket = socket;
        this.in = socket.getInputStream();
    }

    public void interrupt() {
        try {
            socket.close();
        }
        catch (IOException ignored) { }
        finally {
            super.interrupt();
        }
    }

    public void run() {
        try {
            byte[] buf = new byte[BUFSZ];
            while (true) {
                int count = in.read(buf);
                if (count < 0)
                    break;
                else if (count > 0)
                    processBuffer(buf, count);
            }
        } catch (IOException e) { /* Allow thread to exit */ }
    }
}

```

`SocketUsingTask` implements `CancellableTask` and defines `Future.cancel` to close the socket as well as call `super.cancel`. If a `SocketUsingTask` is cancelled through its `Future`, the socket is closed *and* the executing thread is interrupted. This increases the task's responsiveness to cancellation: not only can it safely call interruptible blocking methods while remaining responsive to cancellation, but it can also call blocking socket I/O methods.

7.2. Stopping a Thread-based Service

Applications commonly create services that own threads, such as thread pools, and the lifetime of these services is usually longer than that of the method that creates them. If the application is to shut down gracefully, the threads owned by these services need to be terminated. Since there is

no preemptive way to stop a thread, they must instead be persuaded to shut down on their own.

Sensible encapsulation practices dictate that you should not manipulate a thread—interrupt it, modify its priority, etc.—unless you own it. The thread API has no formal concept of thread ownership: a thread is represented with a `Thread` object that can be freely shared like any other object. However, it makes sense to think of a thread as having an owner, and this is usually the class that created the thread. So a thread pool owns its worker threads, and if those threads need to be interrupted, the thread pool should take care of it.

As with any other encapsulated object, thread ownership is not transitive: the application may own the service and the service may own the worker threads, but the application doesn't own the worker threads and therefore should not attempt to stop them directly. Instead, the service should provide *lifecycle methods* for shutting itself down that also shut down the owned threads; then the application can shut down the service, and the service can shut down the threads. `ExecutorService` provides the `shutdown` and `shutdownNow` methods; other thread-owning services should provide a similar shutdown mechanism.

Provide lifecycle methods whenever a thread-owning service has a lifetime longer than that of the method that created it.

7.2.1. Example: A Logging Service

Most server applications use logging, which can be as simple as inserting `println` statements into the code. Stream classes like `PrintWriter` are thread-safe, so this simple approach would require no explicit synchronization.^[3] However, as we'll see in [Section 11.6](#), inline logging can have some performance costs in high-volume applications. Another alternative is have the `log` call queue the log message for processing by another thread.

[3] If you are logging multiple lines as part of a single log message, you may need to use additional client-side locking to prevent undesirable interleaving of output from multiple threads. If two threads logged multi-line stack traces to the same stream with one `println` call per line, the results would be interleaved unpredictably, and could easily look like one large but meaningless stack trace.

Listing 7.12. Encapsulating Nonstandard Cancellation in a Task with `Newtaskfor` .

```
public interface CancellableTask<T> extends Callable<T> {
    void cancel();
    RunnableFuture<T> newTask();
}

@ThreadSafe
public class CancellingExecutor extends ThreadPoolExecutor {
    ...
    protected<T> RunnableFuture<T> newTaskFor(Callable<T> callable) {
        if (callable instanceof CancellableTask)
            return ((CancellableTask<T>) callable).newTask();
        else
            return super.newTaskFor(callable);
    }
}

public abstract class SocketUsingTask<T>
    implements CancellableTask<T> {
    @GuardedBy("this") private Socket socket;

    protected synchronized void setSocket(Socket s) { socket = s; }

    public synchronized void cancel() {
        try {
            if (socket != null)
                socket.close();
        } catch (IOException ignored) { }
    }

    public RunnableFuture<T> newTask() {
        return new FutureTask<T>(this) {
            public boolean cancel(boolean mayInterruptIfRunning) {
                try {
                    SocketUsingTask.this.cancel();
                } finally {
                    return super.cancel(mayInterruptIfRunning);
                }
            }
        };
    }
}
```

`LogWriter` in [Listing 7.13](#) shows a simple logging service in which the logging activity is moved to a separate logger thread. Instead of having the thread that produces the message write it directly to the output stream, `LogWriter` hands it off to the logger thread via a

`BlockingQueue` and the logger thread writes it out. This is a multiple-producer, single-consumer design: any activity calling `log` is acting as a producer, and the background logger thread is the consumer. If the logger thread falls behind, the `BlockingQueue` eventually blocks the producers until the logger thread catches up.

Listing 7.13. Producer-Consumer Logging Service with No Shutdown Support.

```
public class LogWriter {
    private final BlockingQueue<String> queue;
    private final LoggerThread logger;

    public LogWriter(Writer writer) {
        this.queue = new LinkedBlockingQueue<String>(CAPACITY);
        this.logger = new LoggerThread(writer);
    }

    public void start() { logger.start(); }

    public void log(String msg) throws InterruptedException {
        queue.put(msg);
    }

    private class LoggerThread extends Thread {
        private final PrintWriter writer;
        ...
        public void run() {
            try {
                while (true)
                    writer.println(queue.take());
            } catch (InterruptedException ignored) {
            } finally {
                writer.close();
            }
        }
    }
}
```



For a service like `LogWriter` to be useful in production, we need a way to terminate the logger thread so it does not prevent the JVM from shutting down normally. Stopping the logger thread is easy enough, since it repeatedly calls `take`, which is responsive to interruption; if the logger thread is modified to exit on catching `InterruptedException`, then interrupting the logger thread stops the service.

However, simply making the logger thread exit is not a very satisfying shutdown mechanism. Such an abrupt shutdown discards log messages that might be waiting to be written to the log, but, more importantly, threads blocked in `log` because the queue is full *will never become unblocked*. Cancelling a producer-consumer activity requires cancelling both the producers and the consumers. Interrupting the logger thread deals with the consumer, but because the producers in this case are not dedicated threads, cancelling them is harder.

Another approach to shutting down `LogWriter` would be to set a “shutdown requested” flag to prevent further messages from being submitted, as shown in [Listing 7.14](#). The consumer could then drain the queue upon being notified that shutdown has been requested, writing out any pending messages and unblocking any producers blocked in `log`. However, this approach has race conditions that make it unreliable. The implementation of `log` is a check-then-act sequence: producers could observe that the service has not yet been shut down but still queue messages after the shutdown, again with the risk that the producer might get blocked in `log` and never become unblocked. There are tricks that reduce the likelihood of this (like having the consumer wait several seconds before declaring the queue drained), but these do not change the fundamental problem, merely the likelihood that it will cause a failure.

Listing 7.14. Unreliable way to add shutdown support to the logging service.

```
public void log(String msg) throws InterruptedException {  
    if (!shutdownRequested)  
        queue.put(msg);  
    else  
        throw new IllegalStateException("logger is shut down");  
}
```



The way to provide reliable shutdown for `LogWriter` is to fix the race condition, which means making the submission of a new log message atomic. But we don't want to hold a lock while trying to enqueue the message, since `put` could block. Instead, we can atomically check for shutdown and conditionally increment a counter to “reserve” the right to submit a message, as shown in `LogService` in [Listing 7.15](#).

7.2.2. `ExecutorService` Shutdown

In [Section 6.2.4](#), we saw that `ExecutorService` offers two ways to shut down: graceful shutdown with `shutdown`, and abrupt shutdown with `shutdownNow`. In an abrupt shutdown, `shutdownNow` returns the list of tasks that had not yet started after attempting to cancel all actively executing tasks.

Listing 7.15. Adding reliable cancellation to `LogWriter`.

```

public class LogService {
    private final BlockingQueue<String> queue;
    private final LoggerThread loggerThread;
    private final PrintWriter writer;
    @GuardedBy("this") private boolean isShutdown;
    @GuardedBy("this") private int reservations;

    public void start() { loggerThread.start(); }

    public void stop() {
        synchronized (this) { isShutdown = true; }
        loggerThread.interrupt();
    }

    public void log(String msg) throws InterruptedException {
        synchronized (this) {
            if (isShutdown)
                throw new IllegalStateException(...);
            ++reservations;
        }
        queue.put(msg);
    }

    private class LoggerThread extends Thread {
        public void run() {
            try {
                while (true) {
                    try {
                        synchronized (LogService.this) {
                            if (isShutdown && reservations == 0)
                                break;
                        }
                        String msg = queue.take();
                        synchronized (LogService.this) {
                            --reservations;
                        }
                        writer.println(msg);
                    } catch (InterruptedException e) { /* retry */ }
                }
            } finally {
                writer.close();
            }
        }
    }
}

```

The two different termination options offer a tradeoff between safety and responsiveness: abrupt termination is faster but riskier because tasks may be interrupted in the middle of execution, and normal termination is slower but safer because the `ExecutorService` does not shut down until all queued tasks are processed. Other thread-owning services should consider providing a similar choice of shutdown modes.

Simple programs can get away with starting and shutting down a global `ExecutorService` from `main`. More sophisticated programs are likely to encapsulate an `ExecutorService` behind a higher-level service that provides its own lifecycle methods, such as the variant of `LogService` in [Listing 7.16](#) that delegates to an `ExecutorService` instead of managing its own threads. Encapsulating an `ExecutorService` extends the ownership chain from application to service to thread by adding another link; each member of the chain manages the lifecycle of the services or threads it owns.

Listing 7.16. Logging Service that Uses an `ExecutorService`.

```
public class LogService {
    private final ExecutorService exec = newSingleThreadExecutor();
    ...
    public void start() { }

    public void stop() throws InterruptedException {
        try {
            exec.shutdown();
            exec.awaitTermination(TIMEOUT, UNIT);
        } finally {
            writer.close();
        }
    }
    public void log(String msg) {
        try {
            exec.execute(new WriteTask(msg));
        } catch (RejectedExecutionException ignored) { }
    }
}
```

7.2.3. Poison Pills

Another way to convince a producer-consumer service to shut down is with a *poison pill*: a recognizable object placed on the queue that means “when you get this, stop.” With a FIFO queue, poison pills ensure that consumers finish the work on their queue before shutting down, since any work submitted prior to submitting the poison pill will be retrieved before the pill; producers should not submit any work after putting a poison pill on the queue. `IndexingService` in [Listings 7.17](#), [7.18](#), and [7.19](#) shows a single-producer, single-consumer version of the desktop search

example from [Listing 5.8](#) on page [91](#) that uses a poison pill to shut down the service.

Listing 7.17. Shutdown with Poison Pill.

```
public class IndexingService {
    private static final File POISON = new File("");
    private final IndexerThread consumer = new IndexerThread();
    private final CrawlerThread producer = new CrawlerThread();
    private final BlockingQueue<File> queue;
    private final FileFilter fileFilter;
    private final File root;

    class CrawlerThread extends Thread { /* Listing 7.18 */ }
    class IndexerThread extends Thread { /* Listing 7.19 */ }

    public void start() {
        producer.start();
        consumer.start();
    }

    public void stop() { producer.interrupt(); }

    public void awaitTermination() throws InterruptedException {
        consumer.join();
    }
}
```

Poison pills work only when the number of producers and consumers is known. The approach in `IndexingService` can be extended to multiple producers by having each producer place a pill on the queue and having the consumer stop only when it receives $N_{\text{producers}}$ pills. It can be extended to multiple consumers by having each producer place $N_{\text{consumers}}$ pills on the queue, though this can get unwieldy with large numbers of producers and consumers. Poison pills work reliably only with unbounded queues.

7.2.4. Example: A One-shot Execution Service

If a method needs to process a batch of tasks and does not return until all the tasks are finished, it can simplify service lifecycle management by using a private `Executor` whose lifetime is bounded by that method. (The `invokeAll` and `invokeAny` methods can often be useful in such situations.)

The `checkMail` method in [Listing 7.20](#) checks for new mail in parallel on a number of hosts. It creates a private executor and submits a task for each host: it then shuts down the executor and waits for termination, which occurs when all the mail-checking tasks have completed.^[4]

^[4] The reason an `AtomicBoolean` is used instead of a `volatile boolean` is that in order to access the `hasNewMail` flag from the inner `Runnable`, it would have to be `final`, which would preclude modifying it.

Listing 7.18. Producer Thread for `IndexingService`.

```
public class CrawlerThread extends Thread {
    public void run() {
        try {
            crawl(root);
        } catch (InterruptedException e) { /* fall through */ }
        finally {
            while (true) {
                try {
                    queue.put(POISON);
                    break;
                } catch (InterruptedException e1) { /* retry */ }
            }
        }
    }

    private void crawl(File root) throws InterruptedException {
        ...
    }
}
```

Listing 7.19. Consumer Thread for `IndexingService`.

```
public class IndexerThread extends Thread {
    public void run() {
        try {
            while (true) {
                File file = queue.take();
                if (file == POISON)
                    break;
                else
                    indexFile(file);
            }
        } catch (InterruptedException consumed) { }
    }
}
```

Listing 7.20. Using a Private `Executor` Whose Lifetime is Bounded by a Method Call.

```
boolean checkMail(Set<String> hosts, long timeout, TimeUnit unit)
    throws InterruptedException {
    ExecutorService exec = Executors.newCachedThreadPool();
    final AtomicBoolean hasNewMail = new AtomicBoolean(false);
    try {
        for (final String host : hosts)
            exec.execute(new Runnable() {
                public void run() {
                    if (checkMail(host))
                        hasNewMail.set(true);
                }
            });
    } finally {
        exec.shutdown();
        exec.awaitTermination(timeout, unit);
    }
    return hasNewMail.get();
}
```

7.2.5. Limitations of `ShutdownNow`

When an `ExecutorService` is shut down abruptly with `shutdownNow`, it attempts to cancel the tasks currently in progress and returns a list of tasks that were submitted but never started so that they can be logged or saved for later processing.^[5]

^[5] The `Runnable` objects returned by `shutdownNow` might not be the same objects that were submitted to the `ExecutorService`: they might be *wrapped* instances of the submitted tasks.

However, there is no general way to find out which tasks started but did not complete. This means that there is no way of knowing the state of the tasks in progress at shutdown time unless the tasks themselves perform some sort of checkpointing. To know which tasks have not completed, you need to know not only which tasks didn't start, but also which tasks were in progress when the executor was shut down.^[6]

^[6] Unfortunately, there is no shutdown option in which tasks not yet started are returned to the caller but tasks in progress are allowed to

complete; such an option would eliminate this uncertain intermediate state.

`TrackingExecutor` in [Listing 7.21](#) shows a technique for determining which tasks were in progress at shutdown time. By encapsulating an `ExecutorService` and instrumenting `execute` (and similarly `submit`, not shown) to remember which tasks were cancelled after shutdown, `TrackingExecutor` can identify which tasks started but did not complete normally. After the executor terminates, `getCancelledTasks` returns the list of cancelled tasks. In order for this technique to work, the tasks must preserve the thread's interrupted status when they return, which well behaved tasks will do anyway.

Listing 7.21. `ExecutorService` that Keeps Track of Cancelled Tasks After Shutdown.

```
public class TrackingExecutor extends AbstractExecutorService {
    private final ExecutorService exec;
    private final Set<Runnable> tasksCancelledAtShutdown =
        Collections.synchronizedSet(new HashSet<Runnable>());
    ...
    public List<Runnable> getCancelledTasks() {
        if (!exec.isTerminated())
            throw new IllegalStateException(...);
        return new ArrayList<Runnable>(tasksCancelledAtShutdown);
    }

    public void execute(final Runnable runnable) {
        exec.execute(new Runnable() {
            public void run() {
                try {
                    runnable.run();
                } finally {
                    if (isShutdown()
                        && Thread.currentThread().isInterrupted())
                        tasksCancelledAtShutdown.add(runnable);
                }
            }
        });
    }

    // delegate other ExecutorService methods to exec
}
```

`WebCrawler` in [Listing 7.22](#) shows an application of `TrackingExecutor`. The work of a web crawler is often unbounded, so

if a crawler must be shut down we might want to save its state so it can be restarted later. `CrawlTask` provides a `getPage` method that identifies what page it is working on. When the crawler is shut down, both the tasks that did not start and those that were cancelled are scanned and their URLs recorded, so that page-crawling tasks for those URLs can be added to the queue when the crawler restarts.

`TrackingExecutor` has an unavoidable race condition that could make it yield false positives: tasks that are identified as cancelled but actually completed. This arises because the thread pool could be shut down between when the last instruction of the task executes and when the pool records the task as complete. This is not a problem if tasks are *idempotent* (if performing them twice has the same effect as performing them once), as they typically are in a web crawler. Otherwise, the application retrieving the cancelled tasks must be aware of this risk and be prepared to deal with false positives.

Listing 7.22. Using `TrackingExecutorService` to Save Unfinished Tasks for Later Execution.

```

public abstract class WebCrawler {
    private volatile TrackingExecutor exec;
    @GuardedBy("this")
    private final Set<URL> urlsToCrawl = new HashSet<URL>();
    ...
    public synchronized void start() {
        exec = new TrackingExecutor(
            Executors.newCachedThreadPool());
        for (URL url : urlsToCrawl) submitCrawlTask(url);
        urlsToCrawl.clear();
    }

    public synchronized void stop() throws InterruptedException {
        try {
            saveUncrawled(exec.shutdownNow());
            if (exec.awaitTermination(TIMEOUT, UNIT))
                saveUncrawled(exec.getCancelledTasks());
        } finally {
            exec = null;
        }
    }

    protected abstract List<URL> processPage(URL url);

    private void saveUncrawled(List<Runnable> uncrawled) {
        for (Runnable task : uncrawled)
            urlsToCrawl.add(((CrawlTask) task).getPage());
    }
    private void submitCrawlTask(URL u) {
        exec.execute(new CrawlTask(u));
    }
    private class CrawlTask implements Runnable {
        private final URL url;
        ...
        public void run() {
            for (URL link : processPage(url)) {
                if (Thread.currentThread().isInterrupted())
                    return;
                submitCrawlTask(link);
            }
        }
        public URL getPage() { return url; }
    }
}

```

7.3. Handling Abnormal Thread Termination

It is obvious when a single-threaded console application terminates due to an uncaught exception—the program stops running and produces a stack trace that is very different from typical program output. Failure of a thread in a concurrent application is not always so obvious. The stack trace may be printed on the console, but no one may be watching the console. Also, when a thread fails, the application may appear to continue to work, so its failure could go unnoticed. Fortunately, there are means of both detecting and preventing threads from “leaking” from an application.

The leading cause of premature thread death is `RuntimeException`. Because these exceptions indicate a programming error or other unrecov-

erable problem, they are generally not caught. Instead they propagate all the way up the stack, at which point the default behavior is to print a stack trace on the console and let the thread terminate.

The consequences of abnormal thread death range from benign to disastrous, depending on the thread's role in the application. Losing a thread from a thread pool can have performance consequences, but an application that runs well with a 50-thread pool will probably run fine with a 49-thread pool too. But losing the event dispatch thread in a GUI application would be quite noticeable—the application would stop processing events and the GUI would freeze. `OutOfTime` on [124](#) showed a serious consequence of thread leakage: the service represented by the `Timer` is permanently out of commission.

Just about any code can throw a `RuntimeException`. Whenever you call another method, you are taking a leap of faith that it will return normally or throw one of the checked exceptions its signature declares. The less familiar you are with the code being called, the more skeptical you should be about its behavior.

Task-processing threads such as the worker threads in a thread pool or the Swing event dispatch thread spend their whole life calling unknown code through an abstraction barrier like `Runnable`, and these threads should be very skeptical that the code they call will be well behaved. It would be very bad if a service like the Swing event thread failed just because some poorly written event handler threw a `NullPointerException`. Accordingly, these facilities should call tasks within a `try-catch` block that catches unchecked exceptions, or within a `try-finally` block to ensure that if the thread exits abnormally the framework is informed of this and can take corrective action. This is one of the few times when you might want to consider catching `RuntimeException`—when you are calling unknown, untrusted code through an abstraction such as `Runnable`. [\[7\]](#)

[\[7\]](#) There is some controversy over the safety of this technique; when a thread throws an unchecked exception, the entire application may possibly be compromised. But the alternative—shutting down the entire application—is usually not practical.

Listing 7.23 illustrates a way to structure a worker thread within a thread pool. If a task throws an unchecked exception, it allows the thread to die, but not before notifying the framework that the thread has died. The framework may then replace the worker thread with a new thread, or may choose not to because the thread pool is being shut down or there are already enough worker threads to meet current demand.

`ThreadPoolExecutor` and `Swing` use this technique to ensure that a poorly behaved task doesn't prevent subsequent tasks from executing. If you are writing a worker thread class that executes submitted tasks, or calling untrusted external code (such as dynamically loaded plugins), use one of these approaches to prevent a poorly written task or plugin from taking down the thread that happens to call it.

Listing 7.23. Typical Thread-pool Worker Thread Structure.

```
public void run() {
    Throwable thrown = null;
    try {
        while (!isInterrupted())
            runTask(getTaskFromWorkQueue());
    } catch (Throwable e) {
        thrown = e;
    } finally {
        threadExited(this, thrown);
    }
}
```

7.3.1. Uncaught Exception Handlers

The previous section offered a proactive approach to the problem of unchecked exceptions. The Thread API also provides the

`UncaughtExceptionHandler` facility, which lets you detect when a thread dies due to an uncaught exception. The two approaches are complementary: taken together, they provide defense-in-depth against thread leakage.

When a thread exits due to an uncaught exception, the JVM reports this event to an application-provided `UncaughtExceptionHandler` (see **Listing 7.24**); if no handler exists, the default behavior is to print the stack trace to `System.err`.^[8]

[8] Before Java 5.0, the only way to control the `UncaughtExceptionHandler` was by subclassing `ThreadGroup`. In Java 5.0 and later, you can set an `UncaughtExceptionHandler` on a per-thread basis with `Thread.setUncaughtExceptionHandler`, and can also set the default `UncaughtExceptionHandler` with `Thread.setDefaultUncaughtExceptionHandler`. However, only one of these handlers is called—first the JVM looks for a per-thread handler, then for a `ThreadGroup` handler. The default handler implementation in `ThreadGroup` delegates to its parent thread group, and so on up the chain until one of the `ThreadGroup` handlers deals with the uncaught exception or it bubbles up to the toplevel thread group. The top-level thread group handler delegates to the default system handler (if one exists; the default is none) and otherwise prints the stack trace to the console.

Listing 7.24. `UncaughtExceptionHandler` Interface.

```
public interface UncaughtExceptionHandler {
    void uncaughtException(Thread t, Throwable e);
}
```

What the handler should do with an uncaught exception depends on your quality-of-service requirements. The most common response is to write an error message and stack trace to the application log, as shown in [Listing 7.25](#). Handlers can also take more direct action, such as trying to restart the thread, shutting down the application, paging an operator, or other corrective or diagnostic action.

Listing 7.25. `UncaughtExceptionHandler` that Logs the Exception.

```
public class UEHLogger implements Thread.UncaughtExceptionHandler {
    public void uncaughtException(Thread t, Throwable e) {
        Logger logger = Logger.getAnonymousLogger();
        logger.log(Level.SEVERE,
            "Thread terminated with exception: " + t.getName(),
            e);
    }
}
```

In long-running applications, always use uncaught exception handlers for all threads that at least log the exception.

To set an `UncaughtExceptionHandler` for pool threads, provide a `ThreadFactory` to the `ThreadPoolExecutor` constructor. (As with all thread manipulation, only the thread's owner should change its `UncaughtExceptionHandler`.) The standard thread pools allow an uncaught task exception to terminate the pool thread, but use a `try-finally` block to be notified when this happens so the thread can be replaced. Without an uncaught exception handler or other failure notification mechanism, tasks can appear to fail silently, which can be very confusing. If you want to be notified when a task fails due to an exception so that you can take some task-specific recovery action, either wrap the task with a `Runnable` or `Callable` that catches the exception or override the `afterExecute` hook in `ThreadPoolExecutor`.

Somewhat confusingly, exceptions thrown from tasks make it to the uncaught exception handler only for tasks submitted with `execute`; for tasks submitted with `submit`, *any* thrown exception, checked or not, is considered to be part of the task's return status. If a task submitted with `submit` terminates with an exception, it is rethrown by `Future.get`, wrapped in an `ExecutionException`.

7.4. JVM Shutdown

The JVM can shut down in either an *orderly* or *abrupt* manner. An orderly shutdown is initiated when the last “normal” (nondaemon) thread terminates, someone calls `System.exit`, or by other platform-specific means (such as sending a `SIGINT` or hitting `Ctrl-C`). While this is the standard and preferred way for the JVM to shut down, it can also be shut down abruptly by calling `Runtime.halt` or by killing the JVM process through the operating system (such as sending a `SIGKILL`).

7.4.1. Shutdown Hooks

In an orderly shutdown, the JVM first starts all registered *shutdown hooks*. Shutdown hooks are unstarted threads that are registered with

`Runtime.addShutdownHook`. The JVM makes no guarantees on the order in which shutdown hooks are started. If any application threads (daemon or nondaemon) are still running at shutdown time, they continue to run concurrently with the shutdown process. When all shutdown hooks have completed, the JVM may choose to run finalizers if `runFinalizersOnExit` is `true`, and then halts. The JVM makes no attempt to stop or interrupt any application threads that are still running at shutdown time; they are abruptly terminated when the JVM eventually halts. If the shutdown hooks or finalizers don't complete, then the orderly shutdown process “hangs” and the JVM must be shut down abruptly. In an abrupt shutdown, the JVM is not required to do anything other than halt the JVM; shutdown hooks will not run.

Shutdown hooks should be thread-safe: they must use synchronization when accessing shared data and should be careful to avoid deadlock, just like any other concurrent code. Further, they should not make assumptions about the state of the application (such as whether other services have shut down already or all normal threads have completed) or about why the JVM is shutting down, and must therefore be coded extremely defensively. Finally, they should exit as quickly as possible, since their existence delays JVM termination at a time when the user may be expecting the JVM to terminate quickly.

Shutdown hooks can be used for service or application cleanup, such as deleting temporary files or cleaning up resources that are not automatically cleaned up by the OS. [Listing 7.26](#) shows how `LogService` in [Listing 7.16](#) could register a shutdown hook from its `start` method to ensure the log file is closed on exit.

Because shutdown hooks all run concurrently, closing the log file could cause trouble for other shutdown hooks who want to use the logger. To avoid this problem, shutdown hooks should not rely on services that can be shut down by the application or other shutdown hooks. One way to accomplish this is to use a single shutdown hook for all services, rather than one for each service, and have it call a series of shutdown actions. This ensures that shutdown actions execute sequentially in a single thread, thus avoiding the possibility of race conditions or deadlock between shut-

down actions. This technique can be used whether or not you use shutdown hooks; executing shutdown actions sequentially rather than concurrently eliminates many potential sources of failure. In applications that maintain explicit dependency information among services, this technique can also ensure that shutdown actions are performed in the right order.

Listing 7.26. Registering a Shutdown Hook to Stop the Logging Service.

```
public void start() {
    Runtime.getRuntime().addShutdownHook(new Thread() {
        public void run() {
            try { LogService.this.stop(); }
            catch (InterruptedException ignored) {}
        }
    });
}
```

7.4.2. Daemon Threads

Sometimes you want to create a thread that performs some helper function but you don't want the existence of this thread to prevent the JVM from shutting down. This is what *daemon threads* are for.

Threads are divided into two types: normal threads and daemon threads. When the JVM starts up, all the threads it creates (such as garbage collector and other housekeeping threads) are daemon threads, except the main thread. When a new thread is created, it inherits the daemon status of the thread that created it, so by default any threads created by the main thread are also normal threads.

Normal threads and daemon threads differ only in what happens when they exit. When a thread exits, the JVM performs an inventory of running threads, and if the only threads that are left are daemon threads, it initiates an orderly shutdown. When the JVM halts, any remaining daemon threads are abandoned—`finally` blocks are not executed, stacks are not unwound—the JVM just exits.

Daemon threads should be used sparingly—few processing activities can be safely abandoned at any time with no cleanup. In particular, it is dan-

gerous to use daemon threads for tasks that might perform any sort of I/O. Daemon threads are best saved for “housekeeping” tasks, such as a background thread that periodically removes expired entries from an in-memory cache.

Daemon threads are not a good substitute for properly managing the life-cycle of services within an application.

7.4.3. Finalizers

The garbage collector does a good job of reclaiming memory resources when they are no longer needed, but some resources, such as file or socket handles, must be explicitly returned to the operating system when no longer needed. To assist in this, the garbage collector treats objects that have a nontrivial `finalize` method specially: after they are reclaimed by the collector, `finalize` is called so that persistent resources can be released.

Since finalizers can run in a thread managed by the JVM, any state accessed by a finalizer will be accessed by more than one thread and therefore must be accessed with synchronization. Finalizers offer no guarantees on when or even if they run, and they impose a significant performance cost on objects with nontrivial finalizers. They are also extremely difficult to write correctly.^[9] In most cases, the combination of `finally` blocks and explicit `close` methods does a better job of resource management than finalizers; the sole exception is when you need to manage objects that hold resources acquired by native methods. For these reasons and others, work hard to avoid writing or using classes with finalizers (other than the platform library classes) [EJ Item 6].

[9] See ([Boehm, 2005](#)) for some of the challenges involved in writing finalizers.

Avoid finalizers.

Summary

End-of-lifecycle issues for tasks, threads, services, and applications can add complexity to their design and implementation. Java does not provide a preemptive mechanism for cancelling activities or terminating threads. Instead, it provides a cooperative interruption mechanism that can be used to facilitate cancellation, but it is up to you to construct protocols for cancellation and use them consistently. Using `FutureTask` and the `Executor` framework simplifies building cancellable tasks and services.